

PAFID: Public AI-Generated Food Image Database Pipeline

PAFID is a modular, extensible pipeline for generating, validating, and rating photorealistic food stimuli using Generative AI. This repository provides the tools to extend the existing 350-item canonical database with new cultural or nutritional variants.

Directory Structure

```
PAFID/  
├── src/                                # Core pipeline scripts  
│   ├── generate_stimuli.py            # Generates images and assigns AI labels  
│   │   from a food list  
│   ├── run_qc.py                     # Automated Quality Control & aware ratings  
│   └── prepare_images.py              # Resizes and packages images for  
experiments  
│   ├── rate_images.py                # Conducts blind AI ratings  
│   └── extract_visual_features.py     # Computes low-level visual statistics  
├── data/                             # Input lists and metadata  
│   ├── food_list_initial_seed.csv    # Seed list (Food column only)  
│   ├── Foodpictures_information_dynamic.csv  
│   └── Foodpictures_information_reference.csv  
├── assets/                           # Reference assets  
│   └── plates/                       # Reference plate images for visual  
consistency  
├── rendered_images/                  # Generated high-res images and metadata  
├── resized_images/                  # Experiment-ready images and JS trial  
└── scripts  
    └── requirements.txt              # Python dependencies
```

Setup

1. Download the Code

You can download the code using Git or by downloading a ZIP file.

Option A: Using Git

```
git clone https://github.com/YourUsername/YourRepo.git  
cd YourRepo/PAFID
```

(Note: Replace *YourUsername/YourRepo* with the actual URL to this repository once published.)

Option B: Downloading as a ZIP

1. Click the green **"Code"** button at the top of the GitHub repository page.

2. Select "**Download ZIP**".
3. Extract the ZIP file to your computer.
4. Open your terminal (Mac/Linux) or Command Prompt/PowerShell (Windows).
5. Navigate to the extracted **PAFID** folder using the **cd** command (e.g., **cd Downloads/FoodStimGeneration-main/PAFID**).

2. Set Up a Virtual Environment (Recommended)

To avoid conflicts with other Python projects on your computer, create a virtual environment.

Create the virtual environment:

```
python -m venv venv
```

(If **python** doesn't work, try **python3 -m venv venv**)

Activate the virtual environment:

- **Mac/Linux:**

```
source venv/bin/activate
```

- **Windows (Command Prompt):**

```
venv\Scripts\activate.bat
```

- **Windows (PowerShell):**

```
.\venv\Scripts\Activate.ps1
```

3. Install Dependencies

Once your virtual environment is active (you should see **(venv)** in your command line prompt), install the required packages:

```
pip install -r requirements.txt
```

4. Set Up Google Gemini Access

The pipeline uses **Google Gemini** for image generation and automated food labelling. Two authentication methods are supported — see [GOOGLE_API_SETUP.md](#) for full instructions.

Option A — Vertex AI (Recommended)

Uses your Google Cloud project credentials. No API key required.

```
gcloud auth application-default login
gcloud auth application-default set-quota-project your-project-id
```

Then set these environment variables (add to `~/.zshrc` or `~/.bash_profile` to make permanent):

- **Mac/Linux:**

```
export GOOGLE_CLOUD_PROJECT="your-project-id"
export GOOGLE_CLOUD_LOCATION="global"
export GOOGLE_GENAI_USE_VERTEXAI="True"
```

- **Windows (PowerShell):**

```
[System.Environment]::SetEnvironmentVariable("GOOGLE_CLOUD_PROJECT",
"your-project-id", "User")
[System.Environment]::SetEnvironmentVariable("GOOGLE_CLOUD_LOCATION",
"global", "User")
[System.Environment]::SetEnvironmentVariable("GOOGLE_GENAI_USE_VERTEXA
I", "True", "User")
```

Option B — AI Studio API Key

For quick personal use. Go to [Google AI Studio](#), sign in, and create an API key. Then:

- **Mac/Linux:**

```
export GEMINI_API_KEY="your-gemini-api-key"
```

- **Windows (Command Prompt):**

```
set GEMINI_API_KEY=your-gemini-api-key
```

- **Windows (PowerShell):**

```
$env:GEMINI_API_KEY="your-gemini-api-key"
```

(Optional) If you plan to use OpenAI's models for image generation instead, set your `OPENAI_API_KEY` in the same manner.

Data Management

This repository includes two versions of the master metadata:

- `data/Foodpictures_information_dynamic.csv`: The **working version** that the pipeline updates with new ratings and visual features.
- `data/Foodpictures_information_reference.csv`: A **static copy** of the original study results (351 items) for alignment and reference.

AI-Assigned Labels

Rather than requiring labels to be specified manually in the seed list, the pipeline uses Gemini to assign four labels per food item automatically:

Column	Description
<code>Category_WHO_10</code>	WHO/FAO food category (10 classes: Fruits, Vegetables, Meat, Fish, Dairy and eggs, Bakery wares and cereals, Confectionery and sweets, Beverages, Ready-to-eat savories, Prepared foods)
<code>Category_Simple_6</code>	Simplified food category (6 classes: Fruit, Vegetable, Protein, Grain, Dessert, Dish)
<code>Natural_vs_transformed</code>	Whether the food is Natural (identifiable as a single biological source, even if minimally prepared) or Transformed (substantially altered through processing, combination, or cooking)
<code>Transformation_score</code>	Continuous score 0–100 reflecting degree of processing (0–10 = raw/whole; 85–100 = highly manufactured)

Classification uses `gemini-2.0-flash` via a single text call per food item and does not depend on the generated image.

Usage Pipeline

1. Generate Stimuli

Add new food items to `data/food_list_initial_seed.csv` (one food name per row, `Food` column only). Then run:

```
python src/generate_stimuli.py --limit 5
```

- For each food, Gemini assigns all four labels (see above) before generating the image.
- **Safe Defaults:** By default, the script will **skip** existing PNG files in `rendered_images/`. Use `--overwrite` only if you explicitly wish to replace an image.
- **Outputs:** Images and per-item metadata saved to `rendered_images/stimuli_master.json`.

2. Classify Without Generating Images

To assign or update labels on an existing set of images without regenerating them:

```
python src/generate_stimuli.py --classify-only
```

- Reads food names from `food_list_initial_seed.csv`.
- Calls Gemini text API to assign all four labels per food.
- Updates `rendered_images/stimuli_master.json` in-place, preserving all existing fields (QC ratings, prompts, visual features, etc.).
- Creates a `.bak` backup of the master before writing.
- **Images are never touched.**

Useful flags:

```
python src/generate_stimuli.py --classify-only --food "Apple" # single item
python src/generate_stimuli.py --classify-only --limit 10    # first N items
```

3. Quality Control & Aware Ratings

Verify the generated images and get "aware" AI ratings (where the AI knows the food label):

```
python src/run_qc.py --stimuli-dir rendered_images/
```

- **Merges results** into `data/Foodpictures_information_dynamic.csv`.

4. Blind AI Ratings

Acquire "blind" AI ratings (where the AI only sees the image and must guess what the food is):

```
python src/rate_images.py --stimuli-dir rendered_images/
```

- **Merges results** into `data/Foodpictures_information_dynamic.csv`.

5. Extract Visual Features

Compute low-level visual statistics (luminance, contrast, edge energy, etc.):

```
python src/extract_visual_features.py --stimuli-dir rendered_images/ --merge-canonical
```

- **Merges results** into `data/Foodpictures_information_dynamic.csv`.

6. Prepare for Experiments

Resize images and generate trial metadata:

```
python src/prepare_images.py --stimuli-dir rendered_images/
```

6. Reset Pipeline

If you want to undo your changes and return the database to the 350-item baseline:

```
python src/reset_pipeline.py
```

- **Safety:** This script will ask for confirmation before deleting any non-canonical images or metadata you have generated.

Dataset Schema (Data Dictionary)

The generated `Foodpictures_information_dynamic.csv` contains the following columns, logically grouped:

1. Primary Identifiers

- **filename:** The exact filename of the generated image (e.g., `apple-raw.png`).
- **food:** The primary name/label of the food item.
- **base_food:** The underlying ingredient or dish name before preparation.
- **prep_form:** How the food was prepared (e.g., raw, sliced, prepared).

2. Categorical Labels

- **food_classification:** Legacy high-level food category (e.g., Fruit, Vegetable, Protein).
- **Category_WHO_10:** AI-assigned classification based on the 10 WHO food groups.
- **Category_Simple_6:** AI-assigned simplified 6-group category (e.g., Staples, Vegetables).
- **natural_vs_transformed:** Binary classification: 'Natural' (whole/unprocessed) or 'Transformed' (cooked/milled/processed).
- **Transformation_score:** AI-assigned 1-10 rating of how processed or altered the food is from its natural state.
- **sweet_vs_savory:** Primary flavor profile categorization.

3. Generation Metadata

- **prompt:** The exact text prompt sent to the LLM to generate the image.
- **model:** The specific AI model used for generation (e.g., `gemini-3-pro-image-preview`).
- **seed:** The RNG seed used during generation (if supported/applicable).
- **created:** Unix timestamp of when the image was generated.

- **style_version**: Identifier for the photographic styling parameters used.
- **plate_reference**: Internal reference to any background plate assets utilized.

4. Empirical Human Ground Truth

(Note: These 0-100 scales represent mean human ratings from real psychophysics surveys)

- **human_calorie_density**: Perceived caloric density.
- **human_healthiness**: Perceived healthiness.
- **human_appeal**: Visual appetizingness/appeal.
- **human_sweetness, human_saltiness, human_sourness, human_bitterness, human_savoriness**: Perceived core taste profiles.
- **human_fattiness, human_spiciness**: Perceived mouthfeel and heat.

5. AI Quality Control (Aware)

- **caption**: AI-generated descriptive caption of the image.
- **aware_observed_food**: What the AI identifies in the image when prompted with the target label.
- **aware_observed_prep**: How the AI identifies the preparation state.
- **label_match**: 'match' or 'mismatch' determining if the image successfully represents the intended food.
- **label_confidence**: AI's confidence (0.0 - 1.0) in the label match.
- **portion_size_ok / plate_rim_visible**: Boolean checks ensuring photographic consistency.
- **qc_issues / qc_reasons**: Lists of any visual artifacts or failures flagged by the AI.
- **qc_model / qc_at**: The AI model used for the QC phase and the timestamp.

6. AI Ratings (Aware)

- **aware_ai_***: (Calorie density, healthiness, and all 8 taste profiles on a 0-100 scale). The AI's subjective estimation of the food *when told what the food is*.

7. AI Ratings (Blind)

- **blind_model**: The AI model used for the blind rating phase.
- **blind_observed_food**: The AI's best guess of the food without knowing the target label.
- **blind_guess_similarity**: 0-100 score of how close the blind guess is to the true label.
- **blind_ai_***: (Calorie density, healthiness, and 8 taste profiles). The AI's subjective estimation based *purely on visual appearance*.

8. Low-Level Vision Metrics

(Standard computer vision metrics matching historical *FoodTriplet-Analysis* baselines)

- **ll_mean_luminance, ll_rms_contrast**: Grayscale intensity and standard deviation.
- **ll_lab_L_mean, ll_lab_L_std, ll_lab_a_mean, ll_lab_a_std, ll_lab_b_mean, ll_lab_b_std**: Mean and standard deviation values across the CIE LAB perceptual color space.
- **ll_hsv_s_mean**: Mean saturation from the HSV color space.
- **ll_edge_energy**: Mean magnitude of the Sobel gradients.
- **ll_hog_pc01 - ll_hog_pc10**: The first 10 Principal Components extracted from the Histogram of Oriented Gradients (HOG) features across the dataset.

Extending the Database

To add new foods to the database, follow these steps:

1. **Add to the Seed List:** Append new food names to the bottom of `data/food_list_initial_seed.csv`. The only required column is `Food`.

```
Food
Acai Bowl
Peking Duck
Baklava
```

2. **Generate Images and Classifications:** Run `python src/generate_stimuli.py` (you can use `--limit N` or `--food "Specific Food"` to only generate the new items). The script will automatically use Gemini to classify the food into WHO and Simple categories, determine if it is Natural or Transformed, and then generate the image.
3. **Generate AI Ratings:** Run `python src/run_qc.py --stimuli-dir rendered_images/` followed by `python src/rate_images.py --stimuli-dir rendered_images/`. This will merge the new AI aware and blind ratings into the dynamic CSV without overwriting existing data.
4. **Calculate Visual Features:** Run `python src/extract_visual_features.py --stimuli-dir rendered_images/ --merge-canonical` to compute and append the low-level CV metrics (`ll_`).
5. **(Optional) Add Empirical Human Data:** If you have collected your own human psychophysics ratings, you can integrate them by formatting your aggregate means into a CSV named `data/human_ratings.csv`. The file must contain a `filename` column matching the images, alongside your data (e.g., `human_calorie_density`, `human_healthiness`). Once saved, simply re-run Step 3 (`python src/run_qc.py --stimuli-dir rendered_images/`) and the script will automatically detect and merge your human data into the master dynamic dataset. *(Note: The original `Food survey/` directory used by the authors is excluded from version control for privacy).*

The seed list includes 3 demo items at the end as an example. You can remove them or use them as a template.

Citation

If you use this database or pipeline in your research, please cite: (Insert Citation Here)

Appendix: AI System Prompts

To ensure full transparency, below are the core system prompts utilized by the pipeline to interact with the LLMs.

1. Image Generation Prompt (`generate_stimuli.py`)

```
"A photorealistic studio photograph of {food}, single subject, no packaging, no brand labels or text. Placed on a simple plain white round plate on a plain matte light grey background, even, soft studio lighting. {vessel_clause} Render the image on a square 1024x1024 pixel canvas (1:1 aspect ratio). Do not change the aspect ratio or return a rectangular image. Center the plate within the
```

square frame with equal margins on all sides; ensure the entire plate (and bowl, if present) is fully visible with no cropping. Maintain a FIXED three-quarter camera geometry corresponding to approximately a 40–45° downward tilt from vertical (classic food photography angle). The round plate must appear as an ellipse with a consistent major-to-minor axis ratio across all images. Do NOT vary camera tilt, camera height, focal length, or perspective to better show the food. The camera is positioned above and slightly in front of the plate, angled down toward the centre (not top-down, not side-on). Show a typical single-serving portion size of **{food}** as commonly served to one adult (not a tiny sample and not an oversized platter). Keep the portion neatly arranged and fully visible on the plate with some rim still visible (do not cover the entire plate). The **{food}** is shown in a typical ready-to-eat, edible form appropriate to the requested preparation. The food must be physically grounded on the plate with natural contact shadows and occlusion at the base; avoid white halos, cutout edges, pasted-on appearance, or floating pieces. Ensure consistent lighting direction and shadow softness between the food and the plate. Use realistic surface micro-texture (e.g., pores, fibers, grain separation) and avoid waxy or plastic-looking surfaces. Ensure physically plausible scale and proportions (e.g., slices must match the size of the whole item). High detail, natural colors, minimal shadows, sharp focus, stock-photo style. No logos, no brand names, no watermarks, no text, no human hands, no patterned plates, no cutting boards, no trays, no complex props, no busy backgrounds, no steam to indicate temperature, no multiple items, no multiple servings, no family-style platters, no collages, no extreme side views, no fisheye or distorted perspective, no camera angle changes, no perspective drift, no adaptive framing **{no_bowl_clause}**"

2. Text Classification Prompt (**generate_stimuli.py**)

"Classify the food item **{food}** using all four schemes below.

WHO 10 categories — pick exactly one:

- Dairy and eggs: milk, cheese, yogurt, butter, cream, eggs
- Fruits: fresh, dried, or minimally processed fruit
- Vegetables: fresh, dried, or minimally processed vegetables
- Confectionery and sweets: chocolate, candy, cake, cookies, pastries, ice cream, desserts
- Bakery wares and cereals: bread, crackers, pasta, rice, oats, cereals, grains
- Meat: beef, pork, lamb, poultry, game, and processed meats
- Fish: fish and all seafood
- Beverages: any drink — juice, alcohol, coffee, tea, soda, water
- Ready-to-eat savories: nuts, seeds, crisps, pretzels, popcorn, trail mix — snack foods eaten without further preparation
- Prepared foods: multi-ingredient dishes and meals where no single ingredient dominates

Simple 6 categories — pick exactly one:

- Fruit, Vegetable, Protein, Grain, Dessert, Dish

Natural vs Transformed — pick exactly one:

- Natural: the food is still visually and conceptually identifiable as a single biological food source
- Transformed: the food has been substantially altered from its original biological source

Transformation score — assign a single integer from 0 to 100 using this scale: 0–10: Whole, raw, unmodified (e.g. apple, banana, carrot, tomato) 10–25: Minimally prepared but source-identifiable (e.g. sliced fruit, peeled orange, roasted nuts, dried fruit) 25–40: Simply cooked single-source food (e.g. boiled egg, steamed vegetables, grilled fish, plain rice) 40–55: Mechanically altered single-source food (e.g. mashed potato, minced meat, fruit puree, smoothie) 55–70: Biochemically or structurally transformed (e.g. cheese, yoghurt, tofu, bread, pasta) 70–85: Composite prepared dish (e.g. soup, curry, sandwich, sushi, dumplings) 85–100: Highly transformed or manufactured food (e.g. pizza, cake, sausage, chocolate bar, cereal, candy)

Reply in exactly this format — four lines, no explanation: WHO10: SIMPLE6: NATURAL: SCORE: "

3. Aware AI Ratings / Quality Control ([run_qc.py](#))

System Instruction:

"You are doing neutral, visual quality control for experimental food stimuli. Be factual. Do not mention brands. Do not mention the prompt. Do not add opinions beyond the requested ratings. For any 0–100 ratings (calories/health/flavour), provide best-effort *subjective judgements* based only on what is visually inferable from the image and typical culinary expectations (ingredients, cooking method, portion, sauces). These are not objective measurements. For 'fatty', judge fatty-tasting richness/oiliness/creaminess (mouthfeel), not fat content. If highly uncertain, use 50."

Task Prompt (Image included):

"You will be shown an image of food on a plate. EXPECTED LABELS (from dataset):

- expected_food: {expected_food}
- expected_base_food: {expected_base_food}
- expected_prep_form: {expected_prep_form}
- expected_category: {expected_category}

Tasks:

1. Write a brief neutral caption (1 sentence, ≤ 20 words) describing what is visible.
2. Identify the observed food and observed preparation.
3. Compare observed vs expected and rate label match.
4. Flag obvious visual QC issues.
5. Provide 0–100 ratings as *subjective judgements* of perceived flavour intensity and health attributes."

4. Blind AI Ratings ([rate_images.py](#))

System Instruction:

"You are a neutral observer providing visual assessments of food stimuli. Be factual. Do not mention brands. Do not add opinions beyond the requested ratings. For any 0–100 ratings (calories/health/flavour), provide best-effort *subjective judgements* based only on what is visually inferable from the image and typical culinary expectations. These are not objective measurements. For 'fatty', judge fatty-tasting richness/oiliness/creaminess (mouthfeel), not fat content. If highly uncertain, use 50."

Task Prompt (Image included - No text labels):

"You will be shown an image of food on a plate. Tasks:

1. Identify the food visible in the image.
2. Provide 0–100 ratings as *subjective judgements* of perceived flavour intensity and health attributes (best-effort inferences from visible cues + typical culinary expectations). Return ONLY valid JSON with exactly these keys: observed_food, calorie_density_0_100, healthiness_0_100, sweetness_0_100, saltiness_0_100, sourness_0_100, bitterness_0_100, savoriness_0_100, fatty_flavour_0_100, spiciness_0_100."